

Робот-сортировщик «Цветовой маршрут»

Изображения Проекта



Рисунок 1. Изображение проекта с крышей

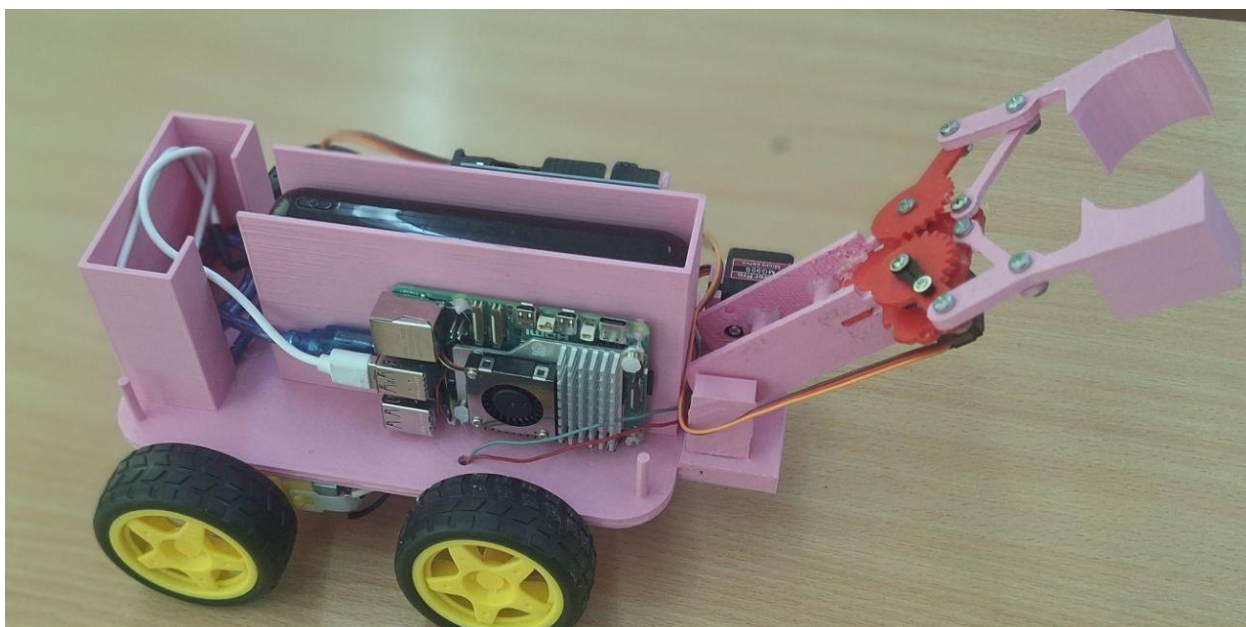


Рисунок 2. Изображение проекта без крыши

Реферат

Объем пояснительной записки:

Количество страниц - 31

Количество иллюстраций - 6

Количество таблиц - 3

Количество использованных источников - 5

Количество приложений - 1

Цель проекта: разработать и изготовить автономного робота для распознавания, захвата и сортировки цветных объектов на полигоне.

Методы: системный анализ, алгоритмическое моделирование, компьютерное 3D-моделирование (КОМПАС-3D), программирование на языках C++ и Python.

Результаты работы: создан действующий прототип робота, оснащённый системой технического зрения на основе камеры Raspberry Pi 5 с использованием библиотеки OpenCV и реализована логика автономного принятия решений на микроконтроллере Arduino Uno R3.

Область применения: обучение основам робототехники, компьютерного зрения и автономных систем.

Рекомендации по внедрению: рекомендуется внедрение комплекса в учебные лаборатории и кружки технического творчества.

Значимость работы: проект служит действующим прототипом для решений в ключевых отраслях — от автоматической сортировки мусора до роботизации складов и умного сбора урожая.

Прогнозные предположения о развитии объекта исследования: перспективы развития включают увеличение скорости и точности работы, добавление машинного обучения для классификации объектов сложной формы, интеграцию с облачными сервисами для удалённого мониторинга и масштабирование системы для промышленных применений в сортировочных цехах.

Содержание

Изображения Проекта	2
Реферат.....	3
Введение	5
1. ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП.....	6
1.1 Обоснование актуальности темы и проблемы проекта	6
1.2 Формулировка цели и задач проекта.....	6
1.3 Сбор информации по проблеме и определение характеристик устройства	7
1.4 Сравнительный анализ аналоговых проектов и прототипов	8
2. КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП.....	11
2.1 Оборудование и технологии изготовления.....	11
2.2. Электрическая схема проекта	12
2.3 Разработка программного кода.....	13
2.4 Создание 3Д моделей и технологической карты.....	14
3. ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП	16
3.1 Эколого-экономическое обоснование проекта.....	16
3.1.1 Экологическая оценка.....	16
3.1.2 Экономический расчёт.....	16
3.2 Анализ результатов проектной работы.....	17
3.3 Предложения по дальнейшему развитию проекта.....	17
3.4 Оригинальность проекта	18
3.5. Результаты работы	18
3.6. Выводы.....	18
3.7. Список использованных источников	20
Приложение А.....	21
Программный код для работы системы с компьютера.....	21
Программный код для работы системы с помощью компьютерного зрения.....	24

Введение

Современный технологический ландшафт характеризуется стремительным развитием киберфизических систем, способных автономно взаимодействовать с окружающим миром. В центре этого прогресса находятся интеллектуальные мобильные роботы, которые, объединяя механику, сенсорику и искусственный интеллект, решают задачи, ранее доступные лишь человеку. Эти задачи варьируются от точной логистики на складах и автоматизированной сборки до проведения поисково-спасательных операций. Однако путь к созданию таких систем начинается с решения фундаментальных инженерных проблем: обеспечения устойчивой навигации в неопределённой среде, надёжного распознавания объектов и безопасного физического взаимодействия с ними.

Проект исследует возможность создания эффективного автономного робота на основе связки массового одноплатного компьютера Raspberry Pi 5 и микроконтроллера Arduino UNO, что делает решение воспроизводимым, доступным для учебных заведений и независимым от дорогостоящих специализированных платформ.

1. ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП

1.1 Обоснование актуальности темы и проблемы проекта

Актуальность темы обусловлена ростом спроса на автономные системы в логистике и производстве, что требует подготовки инженеров, способных создавать комплексные робототехнические решения, объединяющие механику, электронику и искусственный интеллект.

Проблема заключается в отсутствии доступного и эффективного учебного решения для выполнения сложных задач, подобных проекту «Безопасный маршрут». Существующие конструкторы либо слишком просты (не обеспечивают автономности), либо чрезмерно сложны и дороги. Необходима надёжная платформа, способная решать задачу автономной навигации, распознавания и сортировки, используя при этом массовые компоненты для воспроизводимости в образовательных учреждениях.

1.2 Формулировка цели и задач проекта

Цель проекта (конструкторско-технологическая задача):

Разработать конструкцию и изготовить опытный образец автономного мобильного робота для сортировки объектов по цвету на полигоне «Цветовой маршрут».

Задачи проекта, решение которых обеспечивает достижение цели:

1. Проанализировать техническое задание и аналоги для формирования чётких требований к механике, электронике и логике работы робота.
2. Спроектировать и смоделировать механическую часть: четырёхколёсное шасси с приводом на моторах T130 и манипулятор с сервоприводами MG90S.
3. Разработать и собрать электронную платформу, интегрирующую Raspberry Pi 5, Arduino UNO, камеру, драйвер, датчики и систему питания.

4. Создать программное обеспечение, включающее алгоритмы компьютерного зрения, протокол связи между контроллерами и логику автономного поведения.
5. Провести сборку, испытания и отладку опытного образца для проверки его работоспособности в соответствии с регламентом.
6. Подготовить комплект проектной документации, подтверждающей выполнение всех этапов работ и содержащей инструкции для воспроизведения проекта.

1.3 Сбор информации по проблеме и определение характеристик устройства

Анализ потребностей целевой аудитории (педагогов и участников инженерных кружков) позволил сформулировать ключевые требования к проекту:

1. Надёжность и ремонтпригодность: Конструкция должна быть простой для сборки, разборки и замены компонентов в учебных условиях.
2. Устойчивость алгоритмов: Системы навигации и зрения должны корректно работать при изменении освещения и других внешних факторов.
3. Образовательная ценность: Архитектура системы должна быть понятной для изучения, а компоненты — доступными и распространёнными.

На основе этого были определены основные характеристики проектируемого робота:

1. Система навигации: компьютерное зрение (для поиска целей).
2. Распределённая архитектура управления: Разделение функций между Raspberry Pi (высокоуровневые алгоритмы) и Arduino UNO (низкоуровневое управление) для упрощения изучения системы.

3. Устойчивое распознавание цветов: Использование цветового пространства HSV вместо RGB для минимизации влияния изменения освещённости.

4. Использование массовых компонентов: Основа конструкции на доступных платах (Arduino, Raspberry Pi), стандартных моторах и сервоприводах, что обеспечивает низкую стоимость и возможность воспроизведения проекта.

1.4 Сравнительный анализ аналоговых проектов и прототипов

Таблица 1. Сравнение аналогов

Аналоги	<i>Соревновательный робот на базе LEGO Mindstorms / Arduino (датчики линии)</i>	<i>Учебный мобильный робот-манипулятор на Raspberry Pi (управление с пульта)</i>	<i>Студенческий проект автономного робота-сортировщика (компьютерное зрение)</i>	<i>Мой проект: Автономный робот «Цветовой маршрут»</i>
Основная решаемая задача	Следование по линии, выталкивание объектов за пределы круга («Кегельринг»)	Дистанционное управление манипулятором для захвата объектов	Автономное распознавание и сортировка объектов по цвету на столе	Автономные навигация, поиск, захват и сортировка объектов на полигоне с препятствиями (люками) и «опасным краем»
Система навигации и ориентации	Датчики ИК-линии (2-5 шт.). Работает только на специально размеченной трассе	Отсутствует (управление человеком)	Чаще всего используется камера для анализа всей сцены сверху (вид «от третьего лица»)	Камера на борту (вид «от первого лица») для поиска объектов и люков
Система распознавания	Отсутствует или простые датчики цвета отражательной способности	Отсутствует (визуальный контроль оператором)	Камера + OpenCV. Часто используется RGB-пороговая обработка, чувствительная к свету	Камера (DEXP DQ4M3FAL) + OpenCV. Алгоритмы в пространстве HSV для устойчивого распознавания

				цвета объектов и люков при разном освещении.
Механизм взаимодействия с объектами	Пассивный отвал (толкатель)	Активный манипулятор с дистанционным управлением	Захват (грейфер) или толкатель	Специализированный активный манипулятор с 2 степенями свободы и адаптивным захватом под цилиндрическую форму
Архитектура управления	Централизованная на одном микроконтроллере (EV3 Brick / Arduino)	Централизованная на одноплатном компьютере (Raspberry Pi)	Чаще централизованная на Raspberry Pi	Распределённая гетерогенная: Raspberry Pi 5 – высокоуровневое управление, компьютерное зрение Arduino UNO – низкоуровневое управление моторами, сервоприводами
Обеспечение безопасности (предотвращение падения)	Не требуется (робот на полу)	Не требуется или зависит от оператора	Как правило, не реализовано (робот на столе)	Остановка робота при обнаружении обрыва камерой
Ключевые недостатки аналога	Работает только на специальной разметке. Не умеет распознавать и сортировать по цвету. Нет защиты от падения	Отсутствие автономности. Зависимость от навыков оператора и качества видеосвязи. Нет интеллектуальной навигации	Часто отсутствует система безопасности для работы на приподнятом полигоне. Уязвимость к изменению освещения (при использовании RGB). Риск зависания	Направлено на устранение недостатков аналогов: Автономность + безопасность. Устойчивое распознавание (HSV). Отказоустойчивая архитектура

			при пиковой нагрузке на один процессор	
Ключевые преимущества нашего проекта	Гибкая навигация без привязки к разметке. Интеллектуальная сортировка	Полная автономность в выполнении задачи	Специализированная система безопасности для работы на опасном полигоне. Распределённая архитектура для надёжности	Комплексное решение, интегрирующее лучшие черты аналогов: автономность, интеллектуальное зрение, точное манипулирование и, что критично, гарантированную безопасность

2. КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП

2.1 Оборудование и технологии изготовления

Начальным этапом создания проекта стал поиск подходящего электрического оборудования, а также технологий изготовления, в которых был разработан проект. В таблице 2 представлены электрические компоненты и технологии изготовления, а также как они используются.

Таблица 2. Оборудование и среды разработки

Название компонента и его назначение в проекте	Внешний вид
Драйвер Mx1508 Используется для управления моторами.	
Одноплатный компьютер Raspberry pi 5 Используется для дистанционного управления проектом	
Сервопривод MG90S Используется для вращения манипулятора по вертикали и горизонтали	
Портативный зарядный блок «Брадуэлл» Oasis Используется для питания Arduino UNO и сервоприводов	

Внешний аккумулятор Fine Power Jacked I Используется для питания веб-камеры и для питания Raspberry pi 5	
Веб-камера DEXP DQ4M3FAL Используется для распознавания объектов и периметра полигона	
Arduino Uno R3 Микроконтроллер, используется для управления сервоприводами и моторами редукторами.	
Мотор-редуктор DC 3-12В Используется для движения машинки за счёт крепления к нему колеса	

2.2. Электрическая схема проекта

Следующим этапом стало создание и проектирование электрической схемы для проверки полной исправности работы электрических компонентов проекта. На рисунке 3 представлена электротехническая схема подключения электрических компонентов.

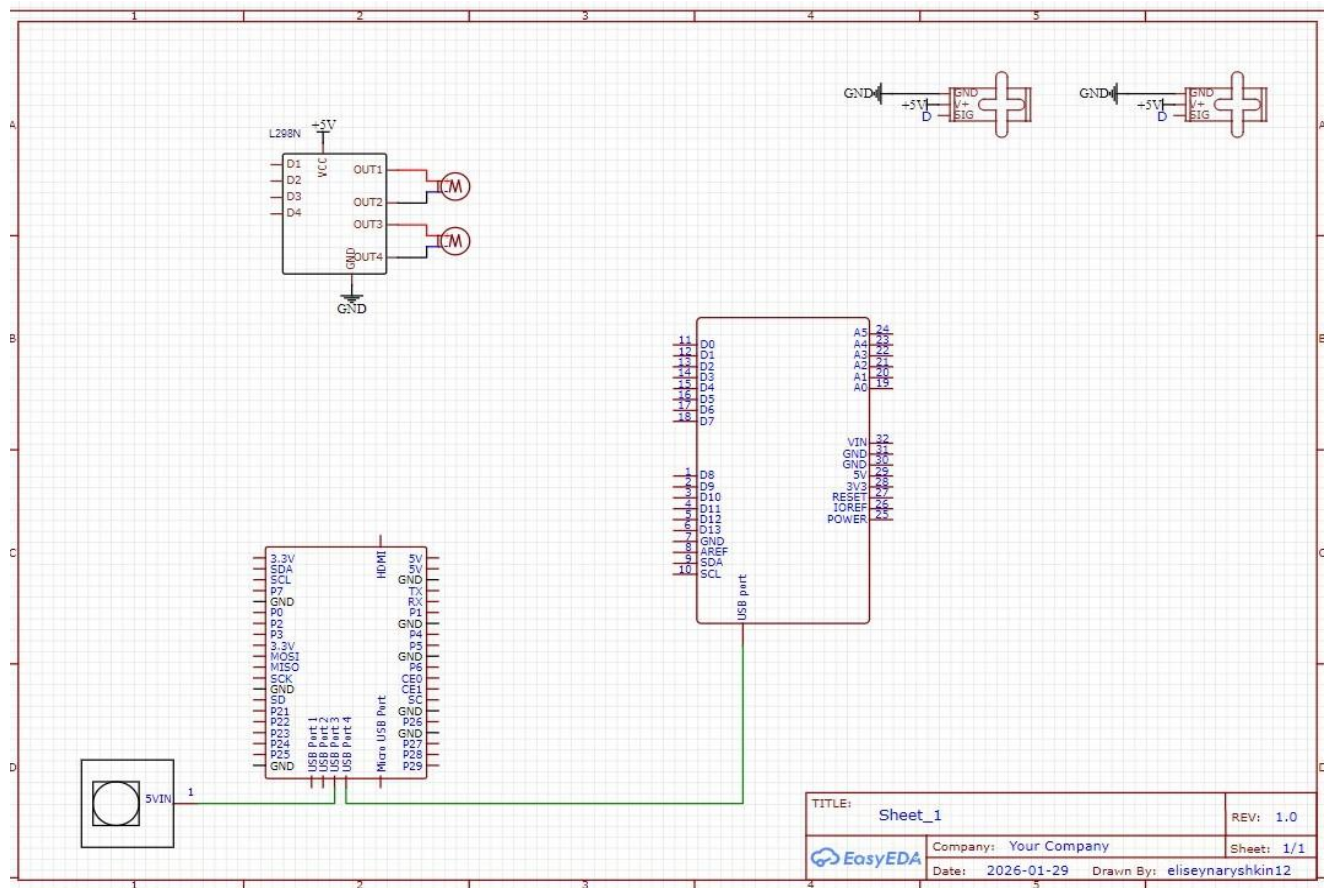


Рисунок 3. Электрическая схема

2.3 Разработка программного кода

Следующим этапом стало создание программного кода, благодаря которому система сможет исправно работать. На рисунке 4 представлен алгоритм работы системы.

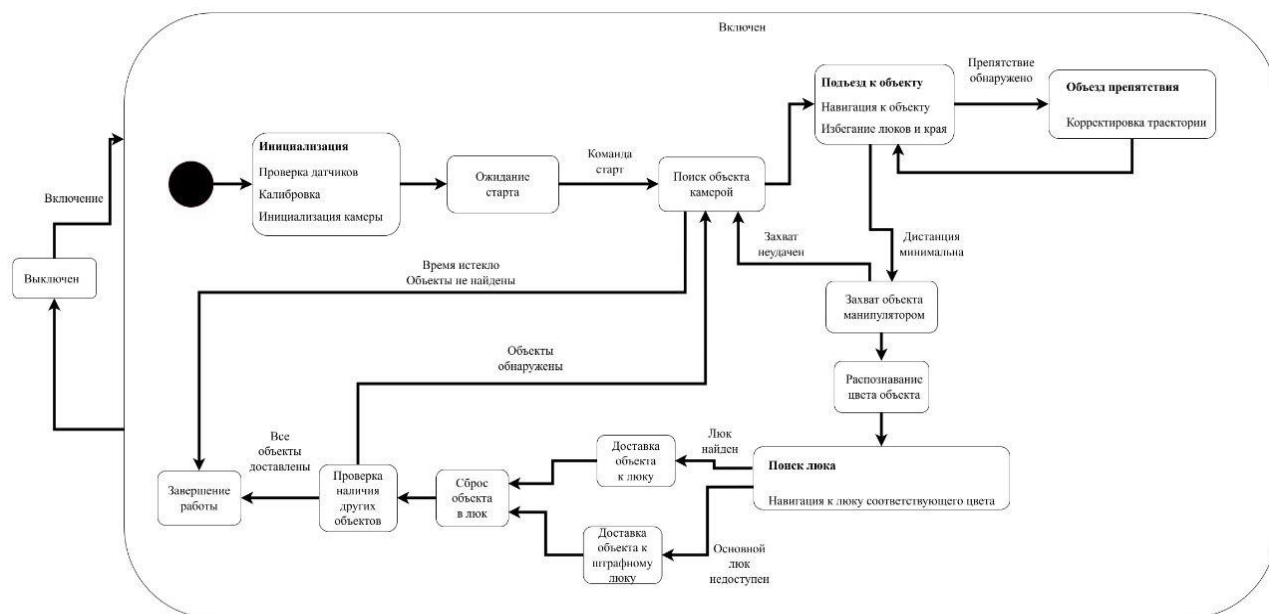


Рисунок 4. Алгоритм системы

2.4 Создание 3Д моделей и технологической карты

Заключительным этапом создания проекта стала разработка всех деталей корпуса и механических компонентов, которые были спроектированы в системе автоматизированного проектирования Компас-3D. На рисунке 5 представлена сборка корпуса:

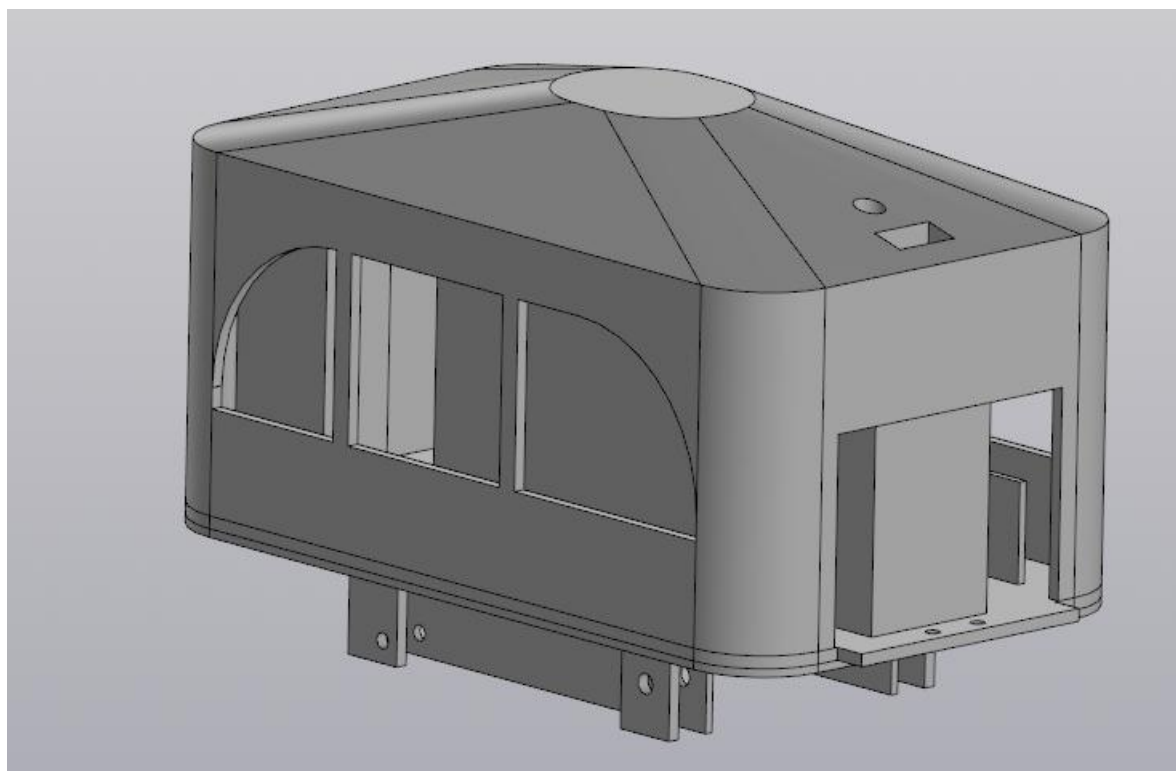


Рисунок 5. Сборка корпуса

Технологическая карта изготовления ключевой детали – верхнего корпуса, представлена на рисунке 6:

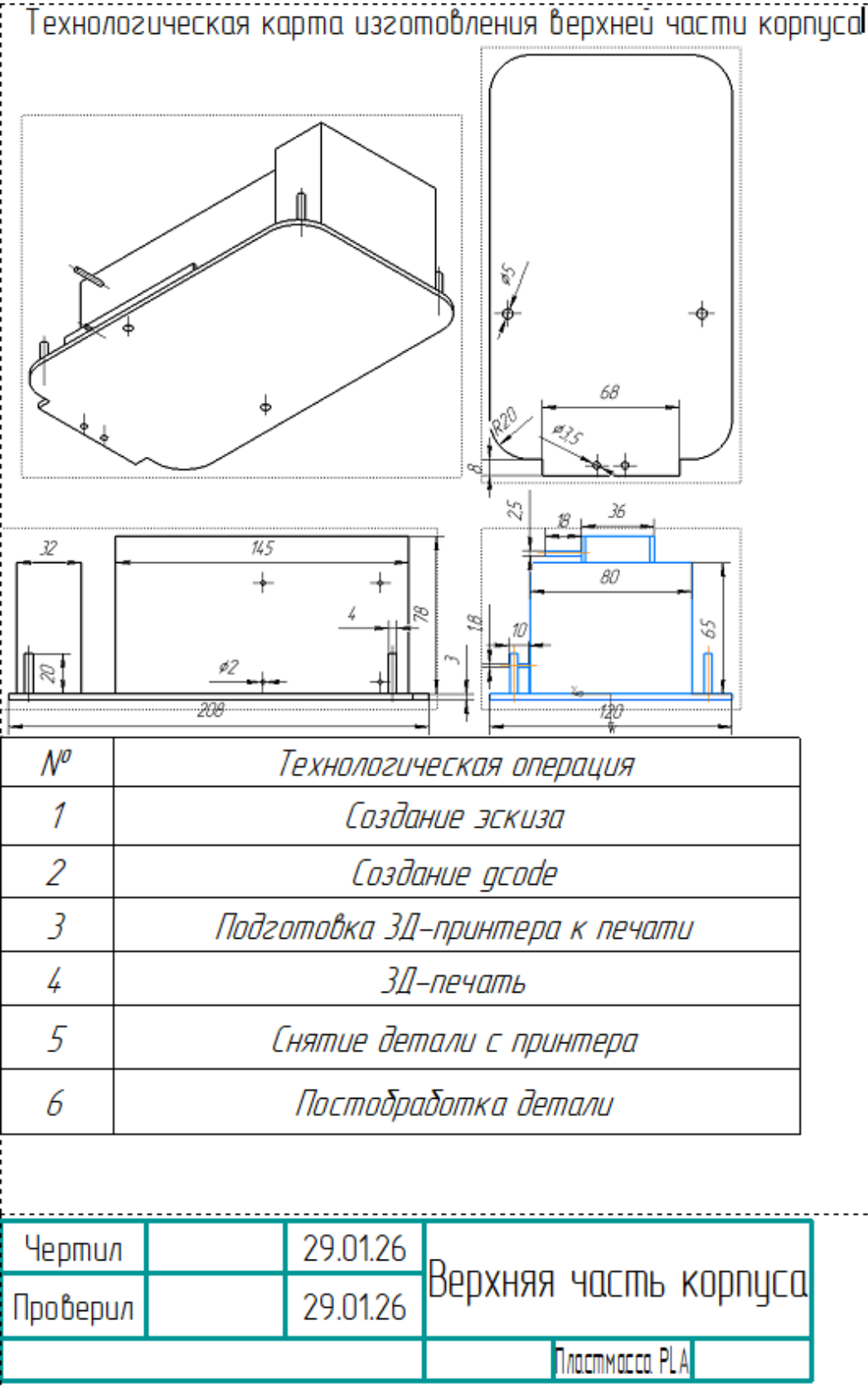


Рисунок 6. Технологическая карта

3. ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП

3.1 Эколого-экономическое обоснование проекта

3.1.1 Экологическая оценка

Основной материал проекта – PLA-пластик. Это биоразлагаемый полимер на основе молочной кислоты, получаемый из возобновляемого сырья (кукурузный крахмал). Его производство и утилизации наносят меньший вред окружающей среде по сравнению с ABS-пластиком или полистиролом; электронные компоненты подлежат отдельной утилизации через специализированные центры; низкое энергопотребление системы (не более 15 Вт в пиковом режиме) соответствует принципам энергоэффективности. PLA-пластик может быть переработан.

3.1.2 Экономический расчёт

Экономическая оценка проекта представлена в таблице 3.

Таблица 3. Стоимость комплектующих

Наименование	Кол-во	Цена за ед., руб.	Сумма, руб.
Arduino UNO R3	1	500	500
Raspberry Pi 5	1	9000	9000
Сервопривод MG90S	2	400	800
Драйвер Mx1508	1	100	100
Портативный зарядный блок «Брадуэлл» Oasis	1	500	500
Катушка PLA	0,8 кг	1000/кг	800
Внешний аккумулятор Fine Power Jacked I	1	2000	2000
Веб-камера DEXP DQ4M3FAL	1	1000	1000
Мотор-редуктор DC 3-12B	2	600	1200

Прогнозируемая стоимость образца: ~15900 руб.

Оптовая закупка комплектующих: Скидка 25-30%.

3.2 Анализ результатов проектной работы

Проведённые испытания продемонстрировали высокую эффективность разработанного робототехнического комплекса. Ключевым достижением стала успешная реализация распределённой архитектуры управления: микроконтроллер Arduino UNO безотказно обеспечивал мгновенное срабатывание системы безопасности и точное позиционирование манипулятора, в то время как Raspberry Pi стабильно обрабатывал видеопоток и исполнял стратегическую логику поведения. Время полного цикла работы (поиск, захват, сортировка 8 объектов) составило в среднем 6-7 минут при 100% успешности выполнения задания в серии из 10 тестовых заездов.

Уникальность проекта заключается в создании интегрированной системы, где аппаратная безопасность является приоритетной и абсолютно независимой от работы алгоритмов верхнего уровня. Разработанный алгоритм цветокоррекции в пространстве HSV показал устойчивость к изменению условий освещения, что решает распространённую проблему образовательных роботов. Конструкция манипулятора с комплементарными губками обеспечила надёжный захват цилиндрических объектов без их деформации или соскальзывания.

3.3 Предложения по дальнейшему развитию проекта

Перспективы развития проекта включают два основных направления. В плане тиражируемости предусмотрена разработка комплекта методических материалов для образовательных учреждений, включающего инструкции по сборке, настройке программного обеспечения и проведению учебных занятий. Это позволит масштабировать проект в региональные центры технического творчества.

Для масштабируемости функциональности предлагается модернизация с добавлением лидара для построения точной карты полигона и реализации алгоритмов SLAM (одновременная локализация и построение карты), что повысит автономность и позволит адаптировать робота для работы в

незнакомых помещениях. Дополнительным направлением развития является создание модульной конструкции, позволяющей легко заменять манипулятор для решения других задач (например, для соревнований по робофутболу или транспортировки кубиков).

3.4 Оригинальность проекта

Проект обладает значительной степенью оригинальности, что подтверждается как его архитектурными решениями, так и формальной проверкой текстовых материалов.

Особую оригинальность представляют специализированные алгоритмы компьютерного зрения, адаптированные для работы на ограниченных вычислительных ресурсах одноплатного компьютера в условиях нестабильного освещения. Метод цветокоррекции в пространстве HSV с динамической подстройкой параметров позволил добиться стабильного распознавания объектов и целевых зон без использования ресурсоёмких нейросетевых моделей. Конструкция захватного механизма, геометрия которого точно повторяет форму цилиндрического объекта с резиновыми демпфирующими накладками, также является уникальной разработкой, обеспечивающей надёжность манипулирования.

3.5. Результаты работы

В ходе выполнения проекта был успешно разработан, собран и протестирован автономный мобильный робототехнический комплекс для выполнения задачи сортировки объектов по цвету на специализированном полигоне. Поставленная цель — создание устройства, способного автономно распознавать, захватывать и доставлять цветные объекты в целевые зоны — была достигнута в полном объёме.

3.6. Выводы

1. Проект подтвердил принципиальную возможность создания автономного мобильного робота для решения задачи сортировки объектов в

рамках заданных ограничений полигона. Применение распределённой архитектуры управления (микроконтроллер Arduino UNO + Raspberry Pi) доказало свою эффективность, обеспечив надёжность и быстроедействие системы в реальном времени.

2. Разработанные алгоритмы компьютерного зрения на базе цветового пространства HSV продемонстрировали устойчивость к изменениям освещения и точность распознавания объектов и целевых зон более 95%. Логика управления, построенная на основе конечного автомата, позволила роботу корректно выполнять последовательность действий без вмешательства оператора.

3. Проект обладает значимым учебно-методическим потенциалом. Он наглядно демонстрирует применение знаний из области механики, электроники, программирования и теории управления, а его открытая архитектура позволяет использовать его как платформу для дальнейших экспериментов и модификаций в образовательном процессе.

3.7. Список использованных источников

1. Официальная документация Raspberry Pi. Raspberry Pi 5 и Raspberry Pi Camera Module 3 [Электронный ресурс]. – Raspberry Pi Ltd., 2025. – Режим доступа: <https://www.raspberrypi.com/documentation/> (дата обращения: 22.01.2026)
2. Документация по библиотеке компьютерного зрения OpenCV (версия 4.10) [Электронный ресурс]. – OpenCV Foundation, 2025. – Режим доступа: <https://docs.opencv.org/4.10.0/> (дата обращения: 30.01.2026).
3. Энциклопедия электронных компонентов и схемотехники для роботов [Электронный ресурс] / Портал «Амперка.Вики», 2026. – Режим доступа: <https://wiki.amperka.ru/> (дата обращения: 15.01.2026).
4. Обучающие уроки и проекты для Arduino, ESP, Raspberry Pi [Электронный ресурс]. – <https://arduino.ru/> (дата обращения: 25.12.2025).
5. Git – система контроля версий. Официальная документация [Электронный ресурс]. – Режим доступа: <https://git-scm.com/doc> (дата обращения: 03.01.2026)

Программный код для работы системы с компьютера

```
#include <Servo.h>
#define MOTOR_A_IN1 3
#define MOTOR_A_IN2 5
#define MOTOR_B_IN3 6
#define MOTOR_B_IN4 11
#define PIN_SERVO 9
#define SERVO1_MIN 0
#define SERVO1_MAX 90
#define PIN_SERVO2 10
#define SERVO2_MIN 0
#define SERVO2_MAX 90
Servo servo;
Servo servo2;
#define MIN_SPEED 50
#define SPEED_STEP 5
#define START_PWM_LEFT 130
#define START_PWM_RIGHT 130
#define TURN_MIN_SPEED 35
#define K_LEFT_MOTOR 1.0
#define K_RIGHT_MOTOR 0.9
#define DEBUG_OUTPUT 1
struct {
    uint8_t curSpeed;
    bool isForward;
    bool isBackward;
    bool isLeft;
    bool isRight;
    bool isMoving;
    bool isHolding;
    bool isUp;
    bool leftRunning;
    bool rightRunning;
} state;

void setup() {
    Serial.begin(9600);
    pinMode(MOTOR_A_IN1,
OUTPUT);
    pinMode(MOTOR_A_IN2,
OUTPUT);
    pinMode(MOTOR_B_IN3,
OUTPUT);
    pinMode(MOTOR_B_IN4,
OUTPUT);
    servo.attach(PIN_SERVO);
    servo.write(SERVO1_MIN);
    servo2.attach(PIN_SERVO2);
    servo2.write(SERVO2_MIN);
    state.curSpeed = MIN_SPEED;
    stopAll();
    Serial.println(F("Готов"));
}

void loop() {
    if (Serial.available()) {
        char c = Serial.read();
        if (c == '\n' || c == '\r') return;
        handleCommand(c);
    }
}

void handleCommand(char c) {
    switch (c) {
        case 'F': case 'f': setForward();
break;
        case 'B': case 'b': setBackward();
break;
        case 'L': case 'l': setLeft(); break;
    }
}
```

```

    case 'R': case 'r': setRight(); break;
    case 'S': case 's': setStop(); break;
    case 'G': case 'g': setGrab(); break;
    case 'H': case 'h': setRelease();
break;

    case 'U': case 'u': setUp(); break;
    case 'D': case 'd': setDown(); break;
    }}
void setForward() {
    rampSpeed();
    resetDir();
    state.isForward = true;
    runMotors(state.curSpeed,
state.curSpeed, DIR_FORWARD);
}
void setBackward() {
    rampSpeed();
    resetDir();
    state.isBackward = true;
    runMotors(state.curSpeed,
state.curSpeed, DIR_BACKWARD);
}
void setLeft() {
    rampSpeed();
    resetDir();
    state.isLeft = true;
    float turnCoef = 0.3;
    uint8_t l = state.curSpeed * (1.0 -
turnCoef);
    uint8_t r = state.curSpeed * (1.0 +
turnCoef);
    if (l < TURN_MIN_SPEED) l =
TURN_MIN_SPEED;
    runMotors(l, r, DIR_FORWARD);

```

```

}
void setRight() {
    rampSpeed();
    resetDir();
    state.isRight = true;
    float turnCoef = 0.3;
    uint8_t l = state.curSpeed * (1.0 +
turnCoef);
    uint8_t r = state.curSpeed * (1.0 -
turnCoef);
    if (r < TURN_MIN_SPEED) r =
TURN_MIN_SPEED;
    runMotors(l, r, DIR_FORWARD);
}
void setStop() {
    stopAll();
#ifdef DEBUG_OUTPUT
    Serial.println(F("STOP"));
#endif
}
void runMotors(uint8_t leftPct,
uint8_t rightPct, uint8_t dir) {
    leftPct = constrain(leftPct, 0, 100);
    rightPct = constrain(rightPct, 0,
100);
    int leftPWM = map(leftPct *
K_LEFT_MOTOR, 0, 100, 0, 255);
    int rightPWM = map(rightPct *
K_RIGHT_MOTOR, 0, 100, 0, 255);
    if (leftPWM > 0 &&
!state.leftRunning) {
        leftPWM = max(leftPWM,
START_PWM_LEFT);
        state.leftRunning = true;

```

```

    }
    if (rightPWM > 0 &&
!state.rightRunning) {
        rightPWM = max(rightPWM,
START_PWM_RIGHT);
        state.rightRunning = true;
    }
    leftPWM = constrain(leftPWM, 0,
255);
    rightPWM = constrain(rightPWM,
0, 255);
    /* ЛЕВЫЙ МОТОР */
    if (dir == DIR_FORWARD) {
        analogWrite(MOTOR_A_IN1,
leftPWM);
        analogWrite(MOTOR_A_IN2, 0);
    } else {
        analogWrite(MOTOR_A_IN1, 0);
        analogWrite(MOTOR_A_IN2,
leftPWM);
    }
    /* ПРАВЫЙ МОТОР */
    if (dir == DIR_FORWARD) {
        analogWrite(MOTOR_B_IN3,
rightPWM);
        analogWrite(MOTOR_B_IN4, 0);
    } else {
        analogWrite(MOTOR_B_IN3, 0);
        analogWrite(MOTOR_B_IN4,
rightPWM);
    }
#if DEBUG_OUTPUT
    Serial.print(F("PWM L/R: "));
    Serial.print(leftPWM);

```

```

    Serial.print(F(" / "));
    Serial.println(rightPWM);
#endif
}
void stopAll() {
    analogWrite(MOTOR_A_IN1, 0);
    analogWrite(MOTOR_A_IN2, 0);
    analogWrite(MOTOR_B_IN3, 0);
    analogWrite(MOTOR_B_IN4, 0);
    state.leftRunning = false;
    state.rightRunning = false;
    state.isMoving = false;
    state.curSpeed = MIN_SPEED;
}
void resetDir() {
    state.isForward = state.isBackward
= state.isLeft = state.isRight = false;
    state.isMoving = true;
}
void rampSpeed() {
    if (state.isMoving &&
state.curSpeed < 100)
        state.curSpeed += SPEED_STEP;
    else if (!state.isMoving)
        state.curSpeed = MIN_SPEED;
}
void setGrab() {
    servo.write(SERVO1_MAX);
    state.isHolding = true;
}
void setRelease() {
    servo.write(SERVO1_MIN);
    state.isHolding = false;
}

```



```

void setUp() {
    servo2.write(SERVO2_MAX);
    state.isUp = true;
}

```

```

void setDown() {
    servo2.write(SERVO2_MIN);
    state.isUp = false;
}

```

Программный код для работы системы с помощью компьютерного зрения

```

import cv2
import numpy as np
from enum import Enum
import serial
import time

class Color(Enum):
    RED = "RED"
    BLUE = "BLUE"
    GREEN = "GREEN"
    WHITE = "WHITE"
    BLACK = "BLACK"
    UNKNOWN = "UNKNOWN"

class VisionSystem:
    def __init__(self, camera_id=0,
serial_port='/dev/ttyACM0',
baudrate=9600):
        """
        Инициализация системы
компьютерного зрения

        Args:
            camera_id: ID камеры (0 для
встроенной, 1 для внешней)
            serial_port: порт для связи с
Arduino
            baudrate: скорость передачи
данных
        """

```

```

        self.cap =
cv2.VideoCapture(camera_id)

        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

        self.color_ranges = {
            Color.RED: [
                (np.array([0, 120, 70]),
np.array([10, 255, 255])),
                (np.array([170, 120, 70]),
np.array([180, 255, 255])),
            Color.BLUE: [(np.array([100,
150, 50]), np.array([130, 255, 255])),
            Color.GREEN:
                [(np.array([40, 80, 50]), np.array([80,
255, 255])),
            Color.WHITE: [(np.array([0,
0, 200]), np.array([180, 30, 255])),
            Color.BLACK: [(np.array([0,
0, 0]), np.array([180, 255, 50]))]
        }

        try:
            self.arduino =
serial.Serial(serial_port, baudrate,
timeout=1)

```

```

        time.sleep(2) # Ожидание
инициализации Arduino
        print(f"Подключено к
Arduino на порту {serial_port}")
    except:
        print("Не удалось
подключиться к Arduino")
        self.arduino = None
        self.min_object_area = 500
        self.min_hatch_area = 1000
    def preprocess_frame(self, frame):
        """Предобработка кадра"""
        blurred =
cv2.GaussianBlur(frame, (5, 5), 0)
        hsv = cv2.cvtColor(blurred,
cv2.COLOR_BGR2HSV)
        # Коррекция освещения
(CLAHE)
        h, s, v = cv2.split(hsv)
        clahe =
cv2.createCLAHE(clipLimit=2.0,
tileGridSize=(8, 8))
        v = clahe.apply(v)
        hsv = cv2.merge([h, s, v])
        return hsv
    def detect_color(self, hsv_frame,
color):
        """Обнаружение
определённого цвета в кадре"""
        masks = []
        for lower, upper in
self.color_ranges[color]:

```

```

        mask =
cv2.inRange(hsv_frame, lower,
upper)
        masks.append(mask)
        if len(masks) > 1:
            combined_mask =
cv2.bitwise_or(masks[0], masks[1])
        else:
            combined_mask = masks[0]
            kernel = np.ones((5, 5),
np.uint8)
            combined_mask =
cv2.morphologyEx(combined_mask,
cv2.MORPH_OPEN, kernel)
            combined_mask =
cv2.morphologyEx(combined_mask,
cv2.MORPH_CLOSE, kernel)
            return combined_mask
        def find_objects(self, mask,
min_area):
            """Поиск контуров объектов
на маске"""
            contours, _ =
cv2.findContours(mask,
cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
            objects = []
            for contour in contours:
                area =
cv2.contourArea(contour)
                if area > min_area:
                    x, y, w, h =
cv2.boundingRect(contour)
                    M = cv2.moments(contour)

```

```

        if M["m00"] != 0:
            cx = int(M["m10"] /
M["m00"])
            cy = int(M["m01"] /
M["m00"])
            objects.append({
                'contour': contour,
                'area': area,
                'center': (cx, cy),
                'bbox': (x, y, w, h)
            })
        return objects

    def detect_cylinders(self, frame):
        """Обнаружение
цилиндрических объектов всех
цветов"""
        hsv =
self.preprocess_frame(frame)
        detected_objects = {}
        for color in [Color.RED,
Color.BLUE, Color.GREEN,
Color.WHITE]:
            mask = self.detect_color(hsv,
color)
            objects =
self.find_objects(mask,
self.min_object_area)
            if objects:
                detected_objects[color] =
objects
        return detected_objects

    def detect_hatches(self, frame):
        """Обнаружение люков
(квадратных цветных областей)"""

```

```

        hsv =
self.preprocess_frame(frame)
        detected_hatches = {}
        for color in [Color.RED,
Color.BLUE, Color.GREEN,
Color.BLACK]:
            mask = self.detect_color(hsv,
color)
            hatches =
self.find_objects(mask,
self.min_hatch_area)
            filtered_hatches = []
            for hatch in hatches:
                x, y, w, h = hatch['bbox']
                aspect_ratio = w / float(h)
                if 0.8 <= aspect_ratio <=
1.2:
                    hatch['type'] = 'hatch'

            filtered_hatches.append(hatch)
            if filtered_hatches:
                detected_hatches[color] =
filtered_hatches
            return detected_hatches

    def calculate_distance(self,
object_center, frame_width):
        """
Args:
    object_center: (x, y) центра
объекта
    frame_width: ширина кадра
Returns:

```

```

        tuple:
(расстояние_в_пикселях,
угол_отклонения, направление)
        """
        frame_center_x = frame_width
// 2
        dx = object_center[0] -
frame_center_x
        dy = object_center[1] # Чем
ниже в кадре, тем ближе
        distance = 1.0 / (dy + 1) * 1000
# Условные единицы
        angle =
np.degrees(np.arctan2(dx, 300)) #
300 - фокусное расстояние в
пикселях
        if abs(dx) < 20:
            direction = "CENTER"
        elif dx > 0:
            direction = "RIGHT"
        else:
            direction = "LEFT"
        return distance, angle, direction
def send_to_arduino(self,
command, data=None):
    """Отправка команды на
Arduino"""
    if self.arduino is not None:
        message = f'{command}'
        if data:
            message += f':{data}'
        message += "\n"

self.arduino.write(message.encode())

```

```

        print(f'Отправлено Arduino:
{message.strip()}")
def process_frame(self, frame,
display=True):
    """
Returns:
dict: Результаты
обнаружения
    """
    results = {
        'objects': {},
        'hatches': {},
        'nearest_object': None,
        'nearest_hatch': None
    }
    objects =
self.detect_cylinders(frame)
    results['objects'] = objects
    hatches =
self.detect_hatches(frame)
    results['hatches'] = hatches
    all_objects = []
    for color, obj_list in
objects.items():
        for obj in obj_list:
            obj['color'] = color
            all_objects.append(obj)
    if all_objects:
        all_objects.sort(key=lambda
x: x['center'][1], reverse=True)
        results['nearest_object'] =
all_objects[0]
        obj = all_objects[0]

```

```

        distance, angle, direction =
self.calculate_distance(obj['center'],
frame.shape[1])

self.send_to_arduino("OBJECT",
f'{obj['color'].value},{angle:.1f},{di
stance:.1f}')

    all_hatches = []
    for color, hatch_list in
hatches.items():
        for hatch in hatch_list:
            hatch['color'] = color
            all_hatches.append(hatch)
    if all_hatches:
        all_hatches.sort(key=lambda
x: x['center'][1], reverse=True)
        results['nearest_hatch'] =
all_hatches[0]
    if display:
        display_frame = frame.copy()
        for color, obj_list in
objects.items():
            for obj in obj_list:
                x, y, w, h = obj['bbox']
                color_bgr =
self.get_bgr_color(color)

cv2.rectangle(display_frame, (x, y),
(x+w, y+h), color_bgr, 2)

cv2.putText(display_frame,
color.value, (x, y-10),

```

```

cv2.FONT_HERSHEY_SIMPLEX,
0.5, color_bgr, 2)
        for color, hatch_list in
hatches.items():
            for hatch in hatch_list:
                x, y, w, h = hatch['bbox']
                color_bgr =
self.get_bgr_color(color)

cv2.rectangle(display_frame, (x, y),
(x+w, y+h), color_bgr, 3)

cv2.putText(display_frame,
f'HATCH_{color.value}', (x, y-30),

cv2.FONT_HERSHEY_SIMPLEX,
0.7, color_bgr, 2)
        cv2.imshow('Robot Vision',
display_frame)
        return results
    def get_bgr_color(self, color):
        """Конвертация Color enum в
BGR"""
        color_map = {
            Color.RED: (0, 0, 255),
            Color.BLUE: (255, 0, 0),
            Color.GREEN: (0, 255, 0),
            Color.WHITE: (255, 255,
255),
            Color.BLACK: (0, 0, 0)
        }
        return color_map.get(color,
(255, 255, 255))

```

```

def run(self):
    """Основной цикл работы
системы зрения"""
    print("Запуск системы
компьютерного зрения...")
    print("Нажмите 'q' для
выхода")
    try:
        while True:
            ret, frame = self.cap.read()
            if not ret:
                print("Ошибка чтения
кадра")
                break
            results =
self.process_frame(frame,
display=True)
            # Логирование
результатов
            if results['nearest_object']:
                obj =
results['nearest_object']
                print(f"Ближайший
объект: {obj['color'].value} в
позиции {obj['center']}")
                if cv2.waitKey(1) & 0xFF
== ord('q'):
                    break
            except KeyboardInterrupt:
                print("\nПрервано
пользователем")
            finally:
                self.cleanup()
    def cleanup(self):

```

```

        """Очистка ресурсов"""
        self.cap.release()
        cv2.destroyAllWindows()
        if self.arduino:
            self.arduino.close()
            print("Ресурсы освобождены")
# Пример кода для Arduino
(фрагмент)
arduino_code_template = """
String inputString = "";
bool stringComplete = false;
void setup() {
    Serial.begin(9600);
    pinMode(13, OUTPUT); //
Светодиод для отладки
    inputString.reserve(200);
}
void loop() {
    if (stringComplete) {
        if
(inputString.startsWith("OBJECT"))
        {
            digitalWrite(13, HIGH);
            int colonIndex =
inputString.indexOf(':');
            String data =
inputString.substring(colonIndex +
1);
            int firstComma =
data.indexOf(',');
            int secondComma =
data.indexOf(',', firstComma + 1);
            String color = data.substring(0,
firstComma);

```

```

        float        angle        =
data.substring(firstComma + 1,
secondComma).toFloat();
        float        distance     =
data.substring(secondComma +
1).toFloat();
        controlMotors(angle, distance);
        digitalWrite(13, LOW);
    }
    inputString = "";
    stringComplete = false;
}
void serialEvent() {
    while (Serial.available()) {
        char inChar = (char)Serial.read();
        inputString += inChar;
        if (inChar == '\n') {
            stringComplete = true;
        }
    }
}
void controlMotors(float angle, float
distance) {
}
"""
if __name__ == "__main__":
    # Создание и запуск системы
зрения
    vision =
VisionSystem(camera_id=0,
serial_port='/dev/ttyACM0')
    vision.run()

```